

SDD 실습 수업 — 투표 앱 처음 만들기

Matt Pocock's Skills(aihero.dev/skills) Main Flow 기반 · 기술 스택: Next.js(App Router) + API Routes + TypeScript + Neon Postgres

🔥 사전 준비 안내: [0.사전준비(handout)]의 Neon 계정/DB 생성, .env.local 에 DATABASE_URL 연결, 빈 Next.js 프로젝트 뼈대(create-next-app)로 npm run dev 가 정상 실행, GitHub Publish repository 까지 수업전 준비 바랍니다.

0. Pocock's Skills가 만들어내는 명세서 종류

오늘 쓸 5개 스킬(/grill-with-docs → /to-spec → /to-tickets → /implement → /code-review)은 각각 서로 다른 "산출물"을 만듭니다. 이걸 먼저 알고 시작하면 각 단계에서 "지금 뭐가 만들어지고 있는지" 헛갈리지 않습니다.

스킬	산출물	저장 위치 / 형식	담기는 내용
/grill-with-docs	CONTEXT.md (용어 집)	저장소 루트	우리 프로젝트만의 용어와 정의. 구현 방법이나 스펙성 문장은 넣지 않음
/grill-with-docs	ADR (Architecture Decision Record, 아키텍처 결정 기록)	docs/adr/ 폴더	"되돌리기 어렵고, 맥락 없이 보면 의아하고, 진짜 트레이드오프가 있는" 결정만 기록. 이 기준을 통과하는 결정이 없으면 이번 세션엔 ADR 이 하나도 안 나올 수도 있음
/to-spec	스펙(Spec) 문서	이슈 트래커(GitHub Issues 등)에 이슈로 발행되거나, 로컬은 .scratch/ 아래 마크다운	문제 정의 → 해결 방향 → 번호 매겨진 유저스토리 → 구현/테스트 관련 결정 → 스코프 제외 항목 → 기타 노트
/to-tickets	티켓(작업 단위) 여러 개	.scratch/<feature>/issues/<번호>-<slug>.md , 또는 GitHub/Linear 네이티브 이슈	선행 티켓(Blocked by), 완료 조건(Acceptance Criteria), 스키마 ~API~UI~테스트를 관통하는 "수직 슬라이스" 범위
/implement	문서 아님 — 코드 + 테스트	코드베이스 자체	스펙/티켓이 정한 내용을 그대로 구현만 함. 이 스킬 자체는 무엇을 만들지 "결정"하지 않고, 새 명세서도 만들지 않음
/code-review	리뷰 리포트	보통 파일로 남기지 않고 대화 응답으로 제공	"컨벤션 준수(Standards)"와 "스펙 부합(Spec)" 두 축을 절대 하나로 합치지 않고 각각 근거를 인용하며 보고

정리하면: /grill-with-docs 는 "용어와 결정"을, /to-spec 은 "무엇을 왜 만드는지"를, /to-tickets 는 "어떤 순서로 얼마씩 만들지"를 문서로 남깁니다. /implement 는 문서가 아니라 실제 코드를 만들고, /code-review 는 그 결과를 다시 문서(스펙)와 비교해 평가합니다.

⚠ 위 표는 각 스킬의 공식 문서를 바탕으로 정리한 내용입니다. 정확한 파일명·포맷은 스킬 버전에 따라 조금씩 다를 수 있습니다.

0-1. /grill-with-docs vs /grill-me — 무엇이 다른가

오늘은 /grill-with-docs 를 씁니다. 이름이 비슷한 /grill-me 와 헷갈리기 쉬워서 미리 짚고 갑니다.

	/grill-me	/grill-with-docs
상태	stateless — 아무 파일도 안 남김	stateful — 코드 저장소를 읽고 결과를 파일로 기록
결과물	없음. 생각이 더 선명해진 상태로 대화가 끝남	CONTEXT.md (용어집), docs/adr/ (결정 기록)이 생성됨
전제 조건	저장소(repo)도, 프로젝트도 필요 없음	실제 코드 저장소를 읽어서 거기에 맞춰 정렬함
쓰기 좋은 상황	아직 코드로 옮기기 전, 아이디어 자체를 혼자 또는 회의에서 다듬고 싶을 때. 소프트웨어가 아닌 아이디어에도 씀	실제 프로젝트에서 방금 내린 결정을 기록으로 남기고, 나중에 다른 사람(또는 다른 세션의 AI)이 그 맥락을 다시 읽을 수 있어야 할 때

두 스킬 모두 내부적으로는 같은 인터뷰 엔진(/grilling)을 쓰지만, /grill-with-docs 만 그 결과를 저장소에 기록으로 남깁니다. 오늘 우리는 실제 Next.js 프로젝트를 만들고, 이때 내린 결정들을 나중(부록 실습)에 다시 참고할 것이기 때문에 /grill-with-docs 를 씁니다.

1. 수업 개요

- 기술 스택: Next.js(App Router) + API Routes(Route Handlers) + TypeScript + Neon Postgres(@neondatabase/serverless , ORM 없이 raw SQL)
- 오늘 다루는 스킬: /grill-with-docs → /to-spec → /to-tickets → /implement → /code-review (Main Flow 5종)
- 오늘의 실습 방식: 스타터 코드 없이, 빈 프로젝트에서 SDD 흐름을 그대로 따라가며 투표 앱을 처음부터 만듭니다. 학생들은 "AI에게 요청 → 명세서 생성 → 티켓 분해 → 구현 → 리뷰"라는 한 사이클을 앱 하나를 완성할 때까지 온전히 경험합니다.
- 오늘 수업에서는 마감시간 지정과 결과 그래프 생성을 하지 않습니다: 결과 화면은 우선 숫자 리스트로 만듭니다. 마감 시간과 결과 그래프는 "이미 있는 앱에 기능 없기"라는 별도 부록(추가 핸드아웃)에서 다룹니다 — 오늘 만든 앱이 바로 그 실습의 출발점이 됩니다.

2. 오늘 만들 앱 — 목표 스펙

아직 아무 코드도 없는 상태에서 시작합니다. 아래는 오늘 수업이 끝났을 때 완성되어 있어야 할 모습의 예입니다. 코딩 에이전트가 약간 다르게 할 수도 있습니다.

DB 스키마

```
create table polls (  
  id uuid primary key default gen_random_uuid(),  
  question text not null,  
  created_at timestamptz not null default now()  
);  
  
create table options (  
  id uuid primary key default gen_random_uuid(),  
  poll_id uuid not null references polls(id) on delete cascade,  
  label text not null,  
  vote_count integer not null default 0  
);
```

페이지 구성

경로	기능
/	전체 투표 목록
/new	투표 만들기 (질문 + 선택지 2~5개 입력)
/polls/\[id]	투표하기 (선택지 클릭 → 제출)
/polls/\[id]/results	결과 보기 (숫자 리스트 — 그래프는 오늘 다루지 않음)

API

엔드포인트	기능
POST /api/polls	투표 생성
GET /api/polls/\[id]	투표 상세 조회
POST /api/polls/\[id]/vote	투표 제출 (body: optionId)

3. 오늘의 요청 시나리오

수업 시작할 때 강사가 이렇게 롤플레이로 제시합니다:

"동아리 회장한테 연락이 왔어요. '우리 동아리에서 쓸 투표 앱을 하나 만들어주면 좋겠어요. 질문 하나 올리면 사람들이 선택지 중 하나를 골라서 투표하고, 결과를 볼 수 있으면 돼요.'"

이 한 문장짜리 요청을 오늘 SDD 5단계로 끌고 가서 실제로 동작하는 앱까지 만듭니다.

4. 타임라인

구간	내용
도입	오늘 목표, 시나리오 제시, 완성본 미리 시연
DB 개념 설명	데이터베이스란 무엇인가 → 관계형 DB 기본 개념 → SQL 맛보기 → Neon·서버리스·환경변수
환경 점검	사전 준비(빈 프로젝트 + 빈 Neon DB 연결) 완료 확인
개념: SDD Main Flow	5개 스킬 각각의 역할과 분기 기준, /grill-me vs /grill-with-docs
실습 1: /grill-with-docs	앱 전체에 대한 요구사항 인터뷰
실습 2: /to-spec	스펙 문서 생성
실습 3: /to-tickets	3개 티켓으로 분해 (만들기 / 투표하기 / 결과보기)
실습 4: /implement — TICKET 1	DB 스키마 + 투표 만들기
실습 4: /implement — TICKET 2	투표하기
실습 4: /implement — TICKET 3	결과 보기
실습 5: /code-review	전체 결과물 리뷰
마무리	회고, 부록 실습(기능 추가) 예고, 배포는 부록 참고

5. DB 개념 설명

A. 왜 데이터베이스가 필요한가

- 화면 상태(예: `useState`)는 보통 새로고침하면 사라집니다. 서버도 켜다 켜면 메모리에 있던 값은 사라집니다.
- 컴퓨터를 꺼도 남아있어야 하는 데이터(투표 결과 같은)는 디스크에 저장해야 합니다.
- 그냥 파일(JSON)에 저장하면 안 될까? → 여러 사람이 동시에 투표하면 파일이 꼬이고, 데이터가 많아지면 검색/집계가 느려집니다. 이런 문제를 해결하려고 만들어진 게 "데이터베이스"입니다.

B. 관계형 데이터베이스 기본 개념

- 테이블은 엑셀 시트라고 생각하면 됩니다. 행(row) 하나가 데이터 하나, 열(column)이 그 데이터의 속성입니다.
- 오늘 우리가 만들 `polls` 테이블: `id`, `question`, `created_at` 세 열.
- `options` 테이블의 `poll_id` 열: "이 선택지가 어느 투표에 속하는지" 가리킵니다. 이렇게 다른 테이블을 가리키는 걸 foreign key(외래키)라고 부릅니다. 투표 하나에 선택지가 여러 개니까 이렇게 표를 나눕니다.
- `id` 열(primary key)이 왜 필요한가: 질문(`question`)이 같은 투표가 두 개 있을 수도 있으니, 절대 겹치지 않는 고유 번호가 각 행마다 필요합니다.
- 오늘은 이 테이블을 우리가 직접 짜지 않고, 실습 1~4를 거쳐 AI가 스펙을 바탕으로 만들어주는 걸 지켜보고, 그 결과를 우리가 직접 DB에 실행하게 됩니다.

C. SQL 맛보기

```
-- 전체 투표 목록 가져오기 (/ 페이지에서 사용)
select * from polls;

-- 새 선택지 추가하기 (/new 페이지에서 사용)
insert into options (poll_id, label) values ('...', '치킨');

-- 투표 수 1 증가시키기 (투표 버튼 눌렀을 때 사용)
update options set vote_count = vote_count + 1 where id = '...';
```

이 세 줄이 오늘 우리가 쓸 SQL의 전부입니다. /implement 단계에서 AI가 이런 코드와 함께 테이블을 만드는 SQL도 자동으로 써줄 텐데, 뭘 하는 코드인지는 알고 넘어가는 게 좋습니다.

D. Neon, 서버리스 DB, 환경변수

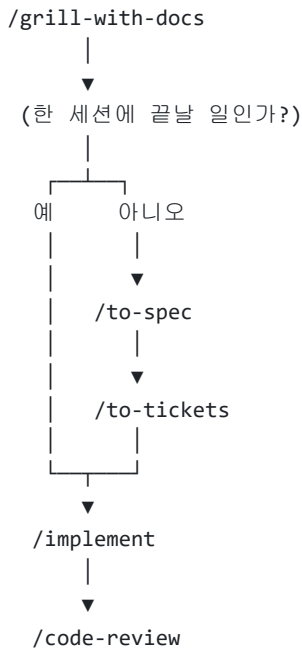
- Neon은 우리 컴퓨터(로컬)가 아니라 인터넷 어딘가에 떠 있는 Postgres 데이터베이스입니다. 설치할 필요 없이 계정만 만들면 바로 씁니다.
- "서버리스"는 안 쓸 때는 꺼져 있다가 요청이 오면 그때 켜지는 방식. 그래서 무료로 쓰기 좋습니다.
- 사전 준비 때 복사한 postgresql://... 문자열은 DB의 주소이자 비밀번호입니다. 이것을 코드 안에 그대로 써넣으면, 코드를 GitHub에 올렸을 때 전 세계 누구나 우리 DB에 접속할 수 있게 됩니다.
- 그래서 이 비밀 문자열은 코드가 아니라 .env.local 이라는 별도 파일에 넣고, 코드에서는 process.env.DATABASE_URL 이라는 이름으로 불러와 씁니다. 이 파일은 GitHub에 올라가지 않도록 .gitignore 에 이미 등록됩니다.
- 사전 준비에서 한 게 바로 이거였습니다 — 여러분 컴퓨터에만 있는 비밀 파일을 만든 것입니다. 다만 오늘은 테이블은 아직 안 만든 "빈 DB" 상태로 시작합니다 — 테이블은 잠시 뒤 실습 1(TICKET 1)에서 함께 만듭니다.

```
// app/api/polls/route.ts (예시)
import { neon } from "@neondatabase/serverless";

const sql = neon(process.env.DATABASE_URL!);

export async function GET() {
  const polls = await sql`select * from polls order by created_at desc`;
  return Response.json(polls);
}
```

6. SDD Main Flow — 언제 어떤 스킬을 쓰나



- `/grill-with-docs` : 모호한 요청을 인터뷰 형식으로 구체화. 합의된 용어([CONTEXT.md](#))와 되돌리기 힘든 결정(ADR)만 남김. (`/grill-me` 와의 차이는 0-1절 참고)
- `/to-spec` : grill 대화를 압축해서 정식 스펙으로 만듦. 여러 세션에 걸쳐 작업해야 할 정도로 클 때만 필요.
- `/to-tickets` : 스펙을 "한 세션 안에 끝낼 수 있는" 단위(티켓)로 쪼갬. 각 티켓은 스키마~API~UI~테스트를 관통하는, 독립적으로 검증 가능한 조각.
- `/implement` : 스펙이나 티켓을 그대로 구현. TDD(실패하는 테스트 → 최소 구현 → 리팩터)로 진행하며, 무엇을 만들지는 스스로 결정하지 않음.
- `/code-review` : 완성된 diff를 "컨벤션 준수"와 "스펙 부합" 두 축으로 나눠 평가. 절대 하나로 합쳐서 점수 매기지 않음.

오늘은 "투표 앱 전체 만들기"라는, 한 세션에 끝내기엔 큰 작업이라 위 흐름의 오른쪽 경로(전체 5단계)를 모두 밟습니다.

7. 실습 상세 — 프롬프트, 결과물, 확인 포인트

⚠ 아래 산출물 예시는 각 스킬의 공식 설명을 바탕으로 미리 구성한 ****예상 형태****입니다. 실제 파일 포맷·문구는 스킬 버전에 따라 다를 수 있으니, 수업 전에 이 시나리오로 한 번 직접 돌려보고 실제 출력으로 이 문서를 교체하시길 권합니다.

실습 1 — `/grill-with-docs`

목표: "투표 앱을 만들어달라"는 한 줄 요청을 실제로 구현 가능한 결정들로 구체화하기

입력 프롬프트

`/grill-with-docs`

투표 앱을 처음부터 만들려고 해. 질문을 하나 올리면 사람들이 선택지 중 하나를 골라 투표하고, 결과를 볼 수 있는 간단한 웹앱이야.

진행 중 에이전트가 물어볼 만한 질문 (학생에게 미리 예고)

- 로그인 없이 익명으로 투표하게 할지?

- 한 사람이 같은 투표에 여러 번 투표하는 걸 막아야 할지, 이번 버전에선 넘어가도 될지?
- 선택지는 몇 개까지 허용할지?
- 결과는 지금 어떤 형태로 보여줄지 (그래프는 이번 버전에서 빼다고 먼저 알려주기)

예상 산출물 1: CONTEXT.md 에 아래와 같은 결정 기록이 생성됨

투표 앱 (Voting App)

동아리, 학생회 등 소규모 조직의 실제 선거 및 의사결정을 지원하는 투표 도구. 운영자가 투표를 올리면 조직 구성원이 선택지 중 하나를 골라 투표하고 결과를 확인한다.

Language

투표(Poll):

조직 구성원에게 결정을 묻기 위해 만들어진 질문과 선택지들의 집합.

Avoid: 설문, 안건, 질문

선택지(Option):

하나의 투표(Poll)에 속하며 투표자가 고를 수 있는 개별 항목.

Avoid: 항목, 보기

투표 행위(Vote):

한 투표자가 특정 투표(Poll)에서 그 투표의 선택 방식(Choice Mode)에 따라 하나 또는 여러 개의 선택지(Option)를 고르는 행위.

Avoid: 표, 응답

선택 방식(Choice Mode):

투표 생성자가 투표(Poll)마다 정하는 설정으로, 단일 선택(하나만 고르기) 또는 복수 선택(여러 개 고르기 허용) 중 하나.

Avoid: 투표 타입, 모드

투표자(Voter):

투표에 참여할 자격이 있는 조직 구성원.

Avoid: 사용자, 참여자

예상 산출물 2: docs/adr/0001-anonymous-voting-shared-operator-token.md

v1은 인증 없는 익명 투표와 운영자 공유 토큰을 사용한다

이 앱은 동아리·학생회의 실제 선거/의사결정에 쓰이지만, v1에서는 투표자 로그인과 중복 투표 방지를 구현하지 않기로 했다 – 투표는 완전 익명이며 같은 사람이 여러 번 투표하는 것을 막을 방법이 없다. 운영자(투표 생성자)도 개별 계정 대신 모든 운영자가 공유하는 단일 비밀번호/토큰으로 구분 없이 인증한다. 이는 "질문을 올리면 사람들이 선택지를 고르고 결과를 본다"는 원래 요청 범위를 단순하게 지키기 위한 의도적 선택이며, 신뢰성보다 단순함을 우선한 트레이드오프다. 투표자 신원을 데이터 모델에 붙이는 작업(로그인, 중복 방지)과 운영자별 개별 계정은 이후 버전에서 별도로 추가할 계획이다.

확인 포인트: 학생 각자 repo에 CONTEXT.md 와 (있다면) ADR 파일이 새로 생겼는지, 자기가 답한 내용이 결정으로 정리되어 있는지 확인. 이 파일들은 부록 실습(기능 추가)에서 AI가 다시 읽고 맥락으로 활용하게 됩니다.

실습 2 — /to-spec

목표: 대화 내용을 "구현 가능한" 스펙 문서로 압축

입력 프롬프트

/to-spec

(직전 grill 대화를 이어서 실행 — 별도 입력 거의 필요 없음)

예상 산출물: .scratch/voting-app/spec.md

< to-spec 완료 메시지 >

스펙을 .scratch/voting-app-mvp/spec.md에 발행했습니다 (ready-for-agent 라벨 적용).

요약: 23개 유저 스토리, Playwright E2E 단일 시임, CONTEXT.md/ADR 두 개를 반영한 구현 결정(스키마, 쿠키 기반 중복 투표 방지, Admin 인증, Server Function 재검증 등), 범위 밖 항목(마감 기능·그래프·다중 Admin 등) 명시.

다음으로 이 스펙을 티켓으로 쪼갤까요(/to-tickets), 아니면 바로 구현에 들어갈까요?

확인 포인트: "스코프 제외" 항목을 학생들과 같이 읽어보며 "오늘은 여기까지만 한다"는 범위를 분명히 하기. "성공 기준"이 테스트로 바로 옮길 수 있을 만큼 구체적인지도 점검.

실습 3 — /to-tickets

목표: 스펙을 "한 세션에 끝낼 수 있는 크기"의 티켓으로 쪼개기

입력 프롬프트

/to-tickets

(직전 스펙 문서를 이어서 실행 — 별도 입력 거의 필요 없음)

예상 산출물: 6개 티켓 (의존 순서대로)

인프라(스키마·DB 연결·테스트 하네스)를 선행 작업으로 두고, 나머지는 실제 사용자 여정을 따라가는 **tracer-bullet** 티켓으로 쪼갭니다. 확인 부탁드립니다.

1. 기반 설정 (Foundational setup)

Blocked by: 없음 (바로 시작 가능)

전달하는 것: polls/options/votes 스키마가 실제(테스트) Neon DB에 만들어지고, DB 연결 헬퍼, 환경변수 배선 (DATABASE_URL/ADMIN_PASSWORD/SESSION_SECRET), Playwright + 테스트 DB 시딩/리셋 하네스가 갖춰짐. 스모크 테스트 하나가 앱 기동+DB 연결을 검증하며 그린.

2. Admin 로그인

Blocked by: 1

전달하는 것: 비밀번호로 로그인, 성공 시 서명된 세션 쿠키 발급(1일 유효), 실패 시 에러 표시, 로그아웃. /admin 경로는 로그인 안 하면 /login으로 리다이렉트되는 일반적인 가드(proxy.ts)가 만들어져서 이후 모든 Admin 페이지가 자동으로 보호됨.

3. Admin이 Poll 생성

Blocked by: 2

전달하는 것: 로그인한 Admin이 질문 + Option 2~10개로 Poll 생성, 서버 단 개수 검증, 생성된 Poll이 Admin용 목록(득표수 0)에 표시됨. 비로그인 상태로 이 경로 접근 시 여전히 리다이렉트되는지도 확인.

4. Voter 투표 + 결과 확인(게이팅)

Blocked by: 3

전달하는 것: 공개 Poll 목록 → Poll 선택 → 무기명 투표 → 쿠키로 중복 투표 차단 → 본인이 투표해야만 결과(득표수 +퍼센트) 노출 → 이미 투표한 Poll 재방문 시 바로 결과로 이동. 여러 Poll 동시 운영도 이 티켓에서 자연히 커버됨(목록에 전부 표시).

5. 결과 실시간(폴링) 갱신

Blocked by: 4

전달하는 것: 결과 화면에서 3~5초 간격으로 최신 득표수를 다시 불러와 새로고침 없이 갱신.

6. Admin이 Poll 삭제

Blocked by: 3

전달하는 것: 삭제 전 확인 단계, 삭제 시 연관 Option·Vote까지 cascade로 함께 삭제, 공개/Admin 목록에서 사라짐. (테스트에서 cascade 검증용 Vote는 DB에 직접 시딩 - 4번 티켓과는 독립적으로 진행 가능.)

확인 포인트: 왜 이 순서로 나뉘었는지? 각 티켓이 이전 티켓의 결과물 위에 쌓이는 "수직 슬라이스"인지? 생각하기

실습 4 — /implement (TICKET 1, 2, 3 순서대로)

/implement 는 TDD(Test-Driven Development)로 동작합니다 — 실패하는 테스트를 먼저 작성 → 통과하는 최소 코드 작성 → 리팩터링 순서로 진행됩니다. 학생은 중간에 터미널에 뜨는 테스트 결과(빨강→초록)를 지켜보는 것만으로도 "TDD가 뭔지" 체감하게 됩니다.

한 TICKET 씩 구현하기

입력 프롬프트

/implement

TICKET 1: DB 스키마(polls, options) 생성 + 투표 만들기 페이지/API

/implement

TICKET 2: 투표하기 페이지/API

/implement

TICKET 3: 결과 보기 페이지/API

TICKET을 모두 한번에 구현하기

/implement

*투표 앱 MVP 구현이 끝났고 커밋 9e2de41로 저장했습니다 (55개 파일, 코드 리뷰에서 나온 재투표 리다이렉트 버그 수정과 중복 코드 정리 포함).

구현 내용

6개 티켓 전부 구현: 기반 설정(Neon 스키마/DB 헬퍼) → Admin 로그인(쿠키 세션, proxy.ts 가드) → Poll 생성(210개 옵션 검증) → Voter 투표 + 게이팅된 결과 → 35초 폴링 실시간 갱신 → Poll 삭제(cascade)

ADR 두 개 준수: ORM 없이 직접 SQL, 계정 없는 쿠키 기반 무기명 투표

Playwright E2E 23개 테스트, 모두 실제 Neon DB 대상으로 통과

코드 리뷰(Standards/Spec 두 축)에서 나온 실제 버그 하나(재투표 시 에러 대신 결과 화면으로 리다이렉트해야 함)를 고치고 회귀 테스트 추가, 중복 코드 2곳(poll 조회 헬퍼, 쿠키 옵션 헬퍼) 정리

아직 안 한 것 (out of scope, 스펙에 명시): Poll 마감 기능, 결과 그래프, 다중 Admin 계정 - 나중에 별도로.

다음에 필요한 것: 배포 전에 .env.local의 ADMIN_PASSWORD를 개발용 placeholder(changeme-dev-password)에서 실제 비밀번호로 바꾸고, Vercel 프로젝트 환경변수에 DATABASE_URL/ADMIN_PASSWORD/SESSION_SECRET을 설정해야 합니다.

실습 5 — /code-review

입력 프롬프트

/code-review

Both reviews are back. Here's the aggregated report:

Standards

No hard violations – proxy.ts, the DAL, and Server Action auth checks all match the Next.js docs patterns verbatim. Four judgement-call smells, all Duplicated Code:

lib/polls.ts: getPollForVoting/getPollResults share an identical "fetch poll or return null" opening. Delete/vote action wiring (.bind(null, id) → useActionState) repeats across delete-poll-button.tsx and vote-form.tsx – only 2 sites, judgement call.
pollId parsing (Number(id) + Number.isInteger guard) duplicated in app/polls/[id]/page.tsx and the results Route Handler.
Cookie-options object duplicated (minus expiry) between lib/session.ts and lib/vote-cookie.ts.
Spec
One real divergence, everything else checks out:

Ticket 04 violation: spec says re-voting should be "차단되고 결과 화면으로 보내짐" (blocked and sent to the results screen), but voteAction in app/polls/[id]/actions.ts just returns an error string when hasVoted is true – it never redirects to results. The page-level guard covers normal navigation, but a stale form resubmit leaves the voter stuck on an error instead of results. Not covered by any test.
MIN/MAX options, 1-day session, cookie dedup + results gating (checked at both the page and the polling API), 3-5s polling with proper cleanup, cascade delete, Server Function re-verification, no ORM, no Voter accounts – all confirmed correct against spec/ADRs.
Summary: Standards 4 findings (all judgement-call duplication, worst: the repeated "fetch poll or null" query shape). Spec 1 finding (the re-vote-redirect gap – worth fixing before commit).

I'll fix the spec gap now and clean up the two most mechanical duplication smells (shared "poll or null" lookup, shared cookie options).

확인 포인트: "⚠ 제안"이 나오면 그걸 바로 반영할지, 다음 과제로 넘길지 판단하게 해보세요 — 이게 리뷰 단계의 핵심(사람이 판단력/taste를 엿는 지점).

8. 마무리

- 오늘 완성한 투표 앱에 같은 방법으로 기능을 추가할 수 있습니다. (참고: 2.부록(기능추가).md)
- 원하는 팀은 부록 실습의 티켓 일부를 /implement 를 AFK(Away From Keyboard — 사람 개입 없이 에이전트가 알아서 진행하는 방식)로 돌려보게 해도 좋습니다.
- 배포는 아래 부록 핸드아웃으로 각자 해보시기 바랍니다.
- 수업 동영상에서 구현한 코드 URL: <https://github.com/junhoege/voting-app> ---

알아두면 좋은 파일 형식

확장자	의미	이 프로젝트에서의 예시
.ts	TypeScript — 화면 없이 로직·타입만 담긴 코드	lib/db.ts , app/api/polls/route.ts
.tsx	TypeScript + JSX — 화면(React 컴포넌트)을 그리는 코드	app/page.tsx , app/new/page.tsx , app/polls/[id]/page.tsx
.test.ts / .test.tsx	위 코드가 의도대로 동작하는지 확인하는 테스트 코드	route.test.ts , page.test.tsx
.sql	데이터베이스에 실행할 SQL 명령어 모음	db/schema.sql (테이블 생성 SQL)
.json	설정값이나 데이터를 담는 형식	package.json (설치한 패키지 목록), tsconfig.json (TypeScript 설정)
.css	화면 디자인(색, 여백, 폰트 등) 정의	app/globals.css (create-next-app 이 기본 생성)
.md	사람이 읽는 문서(마크다운)	CONTEXT.md , specs/voting-app-mvp.md , ADR 파일
.env.local	비밀 값(예: DB 연결 문자열)을 담는 환경변수 파일 (확장자라기보단 파일 이름 자체)	DATABASE_URL=...
.gitignore	Git에 올리지 않을 파일/폴더 목록 (역시 확장자 아닌 파일 이름)	.env.local , node_modules/ 등이 등록됨

정리하면 "화면이 있으면 .tsx , 화면 없이 로직만이면 .ts "가 가장 기본 구분 기준이고, 나머지(.sql , .json , .md , .env.local)는 각각 DB 명령어, 설정, 문서, 비밀값이라는 서로 다른 역할을 맡고 있다고 생각하시면 됩니다.

부록 — Vercel 배포 가이드 (핸드아웃)

준비물

- GitHub 계정
- 오늘 실습한 프로젝트 폴더 (로컬에서 npm run dev 가 정상 동작하는 상태)

1단계 — GitHub에 코드 올리기

Github Desktop을 이용하거나, 터미널에서 프로젝트 폴더로 이동한 뒤 다음 명령어를 입력:

```
git add .
git commit -m "투표 앱 MVP 완성"
git push
```

만약 아직 이 프로젝트를 GitHub 저장소에 연결한 적이 없다면:

1. github.com에서 New repository로 빈 저장소를 하나 만듭니다 (README 체크박스는 끄세요).
2. 터미널에 아래 순서로 입력합니다.

```
git init
git add .
git commit -m "first commit"
git branch -M main
git remote add origin <방금 만든 저장소 주소>.git
git push -u origin main
```

2단계 — Vercel 가입 및 프로젝트 가져오기

1. <https://vercel.com> 접속 → Continue with GitHub로 로그인 (별도 회원가입 필요 없음)
 2. 로그인 후 대시보드에서 Add New... → Project 클릭
 3. Import Git Repository 목록에서 방금 push한 저장소를 찾아 Import 클릭
 4. New Project 화면으로 이동
 5. Application Preset이 자동으로 "Next.js"로 인식되는지 확인 (안 되어 있으면 직접 Next.js 선택)
 6. Create Project 클릭
- 이 단계에서는 아직 Deploy 버튼을 누르지 마세요 — 먼저 DB를 연결합니다.

3단계 — Neon 데이터베이스 연결하기

1. 계속되는 New Project 화면(또는 배포 후 프로젝트 화면)에서 Optional Integrations 탭에서 Neon Storage 확인
2. Add 클릭
3. Link Existing Neon Account 선택 (새로 만들 필요 없음)
4. Integrate Neon 과정 각자 진행
5. 연결하는 순간 DATABASE_URL 같은 환경변수가 자동으로 프로젝트에 추가됩니다. 직접 타이핑할 필요 없습니다.
6. 완료되면 Success! 메시지가 나옵니다.

4단계 — 배포하기

1. 프로젝트 화면으로 돌아가서 Deploy 버튼 클릭
2. 1~2분 정도 빌드 로그가 올라오는 걸 지켜봅니다
3. "Congratulations!" 화면이 뜨면 성공. *.vercel.app 주소로 접속해봅니다 (테이블은 이미 만들어졌으므로 별도 SQL 실행은 필요 없습니다.)
4. 환경변수 셋팅:
 - <https://vercel.com> 로그인 → 해당 프로젝트(voting-app) 선택
 - Settings → Environment Variables
 - ADMIN_TOKEN=Artificial 등록(교수자가 설정한 운영자 로그인 비밀번호)
 - DATABASE_URL도 .env.local에 있는 Neon 연결 문자열로 함께 등록
 - 이 두 값은 로컬 저장소의 .env.local 파일에서 확인 가능
 - 저장 후 Deployments 탭에서 재배포(Redeploy) — 환경 변수는 재배포해야 반영됩니다
5. 배포가 완료되면 URL을 얻게 된다.

(교수자 URL) ?

5단계 — 로컬에서도 같은 DB를 쓰고 싶다면(참고)

```
npm i -g vercel
vercel link
vercel env pull .env.local
```

6단계 — 코드를 고치면 자동으로 재배포됩니다

이후로는 git push 만 하면 Vercel이 자동으로 새 버전을 빌드하고 배포합니다.

문제가 생겼을 때 체크리스트

증상	원인/해결
빌드가 "Build Failed"로 끝남	Vercel 프로젝트 화면 → Deployments → 실패한 빌드 클릭 → 로그의 빨간 글씨(에러 메시지)를 그대로 복사해서 Claude에게 물어보기
사이트는 뜨는데 데이터가 안 보임/에러남	Neon SQL Editor에서 select * from polls; 로 테이블이 있는지 먼저 확인
"환경변수가 없다"는 에러	Storage 탭에서 DB가 실제로 이 프로젝트에 "Connect"되어 있는지 확인. 연결만 하고 재배포를 안 했다면 Deployments 탭에서 최신 배포를 Redeploy
로컬은 되는데 배포본만 안 됨	로컬 .env.local 에만 있고 Vercel에는 없는 환경변수가 있는지 확인 (Vercel 프로젝트 → Settings → Environment Variables)